EPISCOPE

A Big Data Surveillance, Forecasting, and Response-Planning Platform for Bangladesh

Complex Engineering Problem (CEP) Assignment
Technical Report

CSE 4345 · Big Data Analytics
Department of Computer Science & Engineering
Premier University, Chittagong
Spring 2026

Instructor: Rokan Uddin Faruqui

PROJECT TEAM
PROJECT TEAM AND ASSIGNED ROLE

Member	StudentID	Role
Eva Kalam	02222200008101081	Data Engineer
Aklima Akter Naima	02222200008101084	Analytics Engineer
Shreoshee Chowdhury Oma	02222200008101086	Visualization Engineer

Repository: https://github.com/eva550-git/episcope
Video: https://drive.google.com/file/d/1osJj8d93z-F6UhJZgjsqxqTRykccD7l2/view?usp=drive_link

EpiScope: A Big Data Surveillance, Forecasting and Response-Planning Platform for Bangladesh

Group Modulus: Eva Kalam, Aklima Akter Naima, Shreoshee Chowdhury Oma
Dept. of CSE, Premier University, Chittagong, Bangladesh

***Abstract*— EpiScope is an end-to-end teaching prototype that turns routine health-facility encounters, weather, population and stock data into district-level early warnings, one-to-four-week forecasts, a recommended allocation of a fixed national stock and a seasonal preparedness calendar, shown in a privacy-preserving D3.js dashboard. A seeded generator produces synthetic encounters for 64 districts over 2021–2025, driven by real daily weather from Open-Meteo, with negative-binomial counts, 33 injected outbreaks (including a nationwide respiratory surge), duplicate and orphan records, invalid values and a facility that goes silent and then back-fills. At scale factor SF = 0.10 the data set holds 2,699,025 encounter records. Apache Spark ingests them into a bronze–silver–gold lake with SHA-256 file checksums and an idempotent ingest log; 16 quality rules send failing rows to quarantine, patient identifiers are salted and hashed, and the served data hide small counts (8,403 district-week cells suppressed). The dashboard's rolling-origin forecasts had 80% intervals that covered 81.3%, 88.1%, 90.5% and 92.1% of outcomes at horizons of one to four weeks. Quantitative detection results and the full-scale timing runs at SF = 0.25, 0.5 and 1.0 are not included in this version.**

***Index Terms*— disease surveillance, Apache Spark, data quality, privacy, outbreak detection, forecasting, D3.js**

I. Introduction and Problem Analysis

Bangladesh carries a heavy, strongly seasonal burden of infectious disease. Dengue follows the monsoon, and 2023 was the worst season on record; acute watery diarrhoea rises before the monsoon and after floods; influenza-like illness (ILI) peaks in the rainy season; malaria is concentrated in the Chittagong Hill Tracts; and a new epidemic can fill hospitals within weeks. The data needed to act early already exist but are spread over facility reports, laboratories, the meteorological department, the census and district stores, each with its own format, delay and quality. Most analysis is therefore done after the fact, and supplies reach a district after its hospitals are full.

The client brief asks for a platform that, for every district, (i) detects unusual rises in dengue, diarrhoeal disease, ILI and malaria as early as possible, (ii) forecasts cases and hospital demand one to four weeks ahead, (iii) recommends how to allocate a limited national stock, and (iv) produces a seasonal preparedness calendar. Decision-makers are not data scientists, so the platform must explain itself, tolerate late and incomplete data, run on modest hardware and never expose identifiable patients.

We built EpiScope as a three-person project with one lead per role: data engineering (generator, lakehouse, quality, privacy, performance), analytics (detection, forecasting, optimisation, calendar) and visualisation (serving layer and D3.js dashboard). Every requirement of the assignment catalogue is tracked in Appendix D with its current status. This report states only what we measured; items that are not yet measured or not yet integrated are marked in the text and in Appendix D.

TABLE I. Stakeholders, needs and how EpiScope serves them

Stakeholder	Main need / concern	EpiScope feature
MoH policymakers	National picture; defensible allocation	Optimised plan vs population baseline, equity floor
Epidemiologists	Transparent signals; artefacts not mistaken for outbreaks	Adjustable alert threshold, per-load data-quality report
District officers	Early local warning without alert fatigue	Ranked warning table, district forecast, stock
Hospital managers	Demand forecasts with uncertainty	Forecast bands, resource demand view
Vector control	Timing of larval campaigns	Preparedness calendar
Patients, public	No misuse of personal data	Hashed IDs, small-cell suppression
Privacy regulator	Minimal data, small-count suppression	Aggregates only in the browser

II. CEP Attributes and Conflict Resolution

Table II maps the Complex Engineering Problem attributes P1–P7 to concrete decisions in our solution.

TABLE II. How the solution deals with each attribute

Attribute	How it appears in EpiScope
P1 Knowledge	Distributed processing with Spark; columnar Parquet storage and partitioning; data-quality engineering; surveillance statistics and probabilistic forecasting; linear programming for allocation; privacy (salted hashing, small-cell suppression); visual encoding with D3.js.
P2 Conflicts	Five conflicts C1–C5 resolved with numbers where available (below).
P3 Analysis depth	Weather lags, thresholds, horizons and the equity floor are parameters chosen and tested, not fixed by a standard; forecast intervals are checked with a rolling-origin backtest and the allocation is compared with a baseline plan.
P4 Unfamiliar issues	Reporting delays of 0–21 days with fewer reports on Fridays, Saturdays and holidays; a facility silent for 21 days that then back-fills; 1.98% resubmitted records; 0.49% orphan facility keys; ≈1% invalid ages and sexes; Dhaka-dominated skew.

Attribute	How it appears in EpiScope
P5 Beyond codes	No standard turns these data into an allocation plan; we combine WHO IDSR alert logic [8], ICD-10 coding and data-protection principles with our own severity-weighted formulation.
P6 Stakeholders	Seven groups with different needs (Table I); plain-language policy brief (Appendix A).
P7 Interdependence	Quality rules in silver decide what detection and forecasts see; forecasts feed demand and the optimiser; every bronze row keeps its source file, checksum and batch id so a bad input can be traced.

Activity attributes. A1 (resources): two weeks, three people and an 8 GB laptop; this is below the 16 GB the assignment assumes, so the full scale sweep needs another machine (Section VI-D). A2 (interaction): generator, lake, analytics and dashboard share one schema contract and one run script. A3 (innovation): climate, clinical encounters and stock are combined in one allocation model with an explicit equity floor. A4 (consequences): a missed outbreak or a wrong shipment costs lives, so uncertainty and suppression are shown, not hidden. A5 (familiarity): course tools applied to epidemiology, where we had to learn reporting delays and interval scoring.

A. Resolution of conflicts C1–C5

C1 sensitivity vs. false alarms. The alert threshold is adjustable from the dashboard, so epidemiologists can trade earlier warnings against more false alarms; the default requires a signal of at least 2.0 standard deviations together with at least 50% growth. A quantitative trade-off curve against the injected outbreaks is not included in this version.

C2 granularity vs. privacy. Counts of 1–4 identify people, so the browser receives district × epi-week aggregates only and hides those counts. In the SF = 0.10 gold mart 8,403 district-week cells are suppressed. Forecast demand and dispatch values below 5 are also shown as "Suppressed (1–4)" in the allocation table. We have not yet audited whether a hidden cell could be recovered by subtracting published totals.

C3 efficiency vs. equity. Allocation is a linear programme (PuLP/CBC) that minimises severity-weighted unmet demand under the stock and budget limits, with an equity floor α times each district's population share of every resource. The dashboard compares the optimised plan with a population-proportional baseline across budget levels. The what-if panel varies the budget (a share of the replacement cost of the four-week forecast demand), the equity floor α and the alert threshold, and an efficiency–equity chart compares both plans across budget levels from 20% to 100%. A larger α guarantees every district a larger minimum share at some cost in efficiency.

C4 freshness vs. cost. Decisions such as weekly forecasts and shipments do not need minute-level data, so the main cadence is a daily incremental batch; ingestion skips files it has already seen (Section VI-E). No streaming path is part of this version; processing is batch.

C5 accuracy vs. interpretability. Forecasts are produced per district and disease and shown with their intervals next to the observed series, so officials can judge them for their own district. A comparison with a more complex model is not part of this version.

III. Architecture and Design Decisions

A. Layers and tools

Fig. 1 shows the layers. Raw sources are copied unchanged into a bronze layer; silver validates, de-identifies and conforms them; gold holds weekly district × disease marts; analytics read only gold; serving exports small aggregated JSON files that the D3.js dashboard loads. Heavy transformations run in PySpark 3.5.4 in local mode; pandas is used for small reference data and final exports; some silver and gold steps also still use pandas, which Section IX lists as a limitation. Layers are stored as Parquet. We did not use Delta Lake, so there is no ACID MERGE or time travel (ING-7, optional).

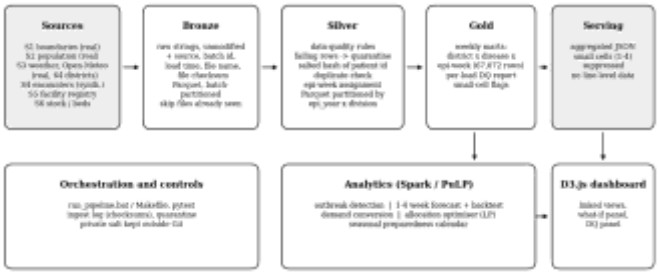


Fig. 1. EpiScope architecture: bronze–silver–gold lake, analytics and the D3.js serving layer.

B. Partitioning

Silver encounters are partitioned by epi-year and division. Almost every query restricts the period, and the allocation and division views restrict the division, so five years and eight divisions give about 40 directories whose files stay large. Partitioning by district would give hundreds of small files and heavy skew, because Dhaka holds a large share of the rows; partitioning by epi-week would give hundreds of tiny files. The cost is that a late report for an old week rewrites its whole year-division partition.

C. Reproducibility

On Windows, one script (run_pipeline.bat) checks Java and then runs generation, ingestion, silver and the later stages in order, one process per stage. On first run it also creates a virtual environment, installs the pinned dependencies and creates a private local salt; the scale factor is set with an environment variable (default SF = 0.10), and a Makefile exists for other platforms. Nothing hard-codes dates or file names: sources are recognised from file headers and keys. The test suite (49 tests passed with Java 8 in the last run recorded in the README) covers quality rules, de-identification and the optimiser.

TABLE III. Alternatives considered

Decision	Chosen	Alternative and why not
Table format	Parquet	Delta/Iceberg: ACID merge and time travel, but heavier set-up on a laptop with slow internet
Storage	Local file system	HDFS or MinIO: more memory than an 8 GB laptop allows; not implemented
Orchestration	Run script + Makefile	Airflow/Prefect: extra services for a small DAG
Solver	PuLP with CBC	Heuristics: no optimality guarantee

Decision	Chosen	Alternative and why not
Serving	Static JSON files	Database API: not needed for a few MB of aggregates

IV. Data Generation, Quality, Lineage and Privacy

A. Public data

District boundaries (64 districts in eight divisions) come from geoBoundaries [10]. District population (total 170,348,385 for 2023) is the denominator for incidence and comes from the project reference data (a single 2023 district estimate). Daily weather (temperature, rainfall, humidity) for 2021-01-01 to 2025-12-31 comes from the Open-Meteo historical archive [9], queried at each district centroid: 116,864 rows (64 districts × 1,826 days). Each district's response is cached, so an interrupted download resumes where it stopped. Simplified boundaries and a weather snapshot are kept in the repository so that a clean clone needs no downloads.

B. Synthetic generator (S4–S6)

Three seeded scripts build a facility registry, daily true counts and encounter-level records; a given seed always gives the same output. The registry has 640 facilities in all 64 districts (354 clinics, 187 upazila health complexes, 99 hospitals; 35,159 beds, 1,723 ICU beds), with capacity growing with district population. Daily counts per district and disease are negative-binomial (over-dispersed), not Poisson. Dengue responds to the rainfall and temperature of the 28-day window that ended four weeks earlier, is concentrated in large cities, and is about 2.3 times higher in 2023 than in a normal year; diarrhoea is raised by 80% for two weeks, starting three days after extreme rain (a three-day total above the district's 95th percentile); ILI and malaria follow their own seasons, with malaria concentrated in the hill districts.

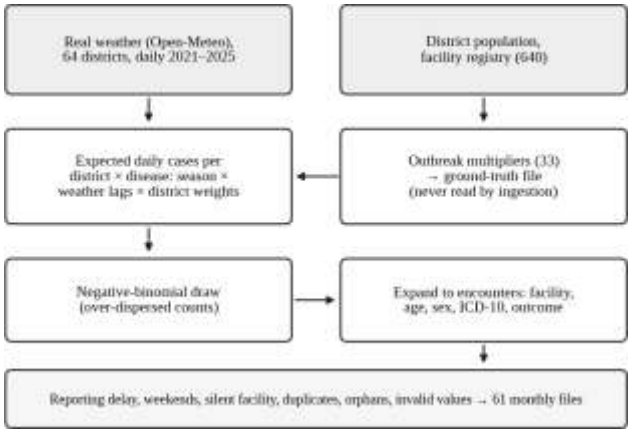


Fig. 2. Synthetic data generation: weather-driven expected counts, injected outbreaks, over-dispersed draws and realistic reporting defects.

We injected 33 outbreaks: 31 sharp local ones (11 dengue, 8 diarrhoea, 8 ILI, 4 malaria), one slow-growing dengue outbreak, and one nationwide ILI surge that starts in Dhaka in March 2022 and reaches other districts up to three weeks later (96 district-level events in all). The truth file lives in a separate folder that ingestion never reads. At SF = 0.10 the daily counts total 101,643 dengue, 805,291 diarrhoea, 693,056 ILI and 17,787 malaria cases (Table IV; the 2023 dengue peak is visible).

TABLE IV. Synthetic cases per year at SF = 0.10

Disease	2021	2022	2023	2024	2025
Dengue	13,739	15,114	37,872	19,455	15,463
Diarrhoea	162,079	159,842	158,559	165,385	159,426
ILI	130,387	171,598	129,931	130,957	130,183
Malaria	3,734	3,557	3,366	3,696	3,434

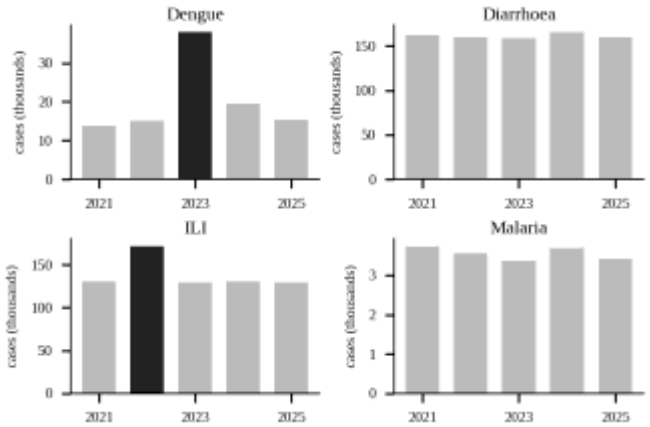


Fig. 3. Synthetic cases per year at SF = 0.10; the 2023 dengue season and the 2022 ILI surge are highlighted.

Daily counts are then expanded into 2,699,025 encounter records (61 monthly files, split by report month; background non-target encounters are included). Realism defects are injected on purpose: 53,384 resubmitted records (1.98%), 26,112 repeat visits of the same patient at another facility within a week, 13,107 facility keys that are not in the registry (0.49%), about 1% invalid ages or sexes, a report delay of 0–21 days (right-skewed) with 70% of Friday, Saturday and holiday submissions pushed one to three days later, and one facility (F0172) that is silent from 2024-06-03 for 21 days and then back-fills 579 reports on 2024-06-24. The encounter generator took 79–88 s at SF = 0.10 on our laptop, and two full runs gave identical monthly row counts and defect counts. Stock and bed-occupancy snapshots (S6) were not part of the ingested data in this version, so the allocation view relies on the stock assumptions stated on the dashboard.

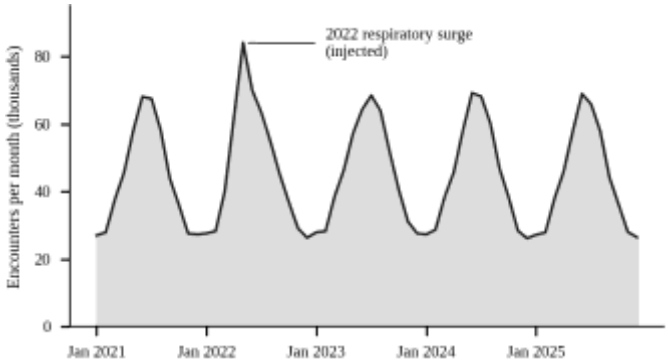


Fig. 4. Encounter records generated per month (2,699,025 in total); the winter and rainy-season cycles and the 2022 surge are visible.

C. Ingestion and incremental loads

Bronze keeps every value as the original text and adds the source, batch id, load time, file name and file checksum. Sources are recognised from headers and keys, and anything in the ground-truth folder is skipped. A log of SHA-256 checksums makes ingestion idempotent (Fig. 5): the first run ingested 61 encounter files (2,699,025 rows), the registry (640 rows), weather (116,864 rows), population (64 rows) and the boundary file (64 features); an immediate second run reported 65 files, 0 new, 65 already ingested, and did nothing. Handling of late records in silver and gold is not demonstrated in this version.

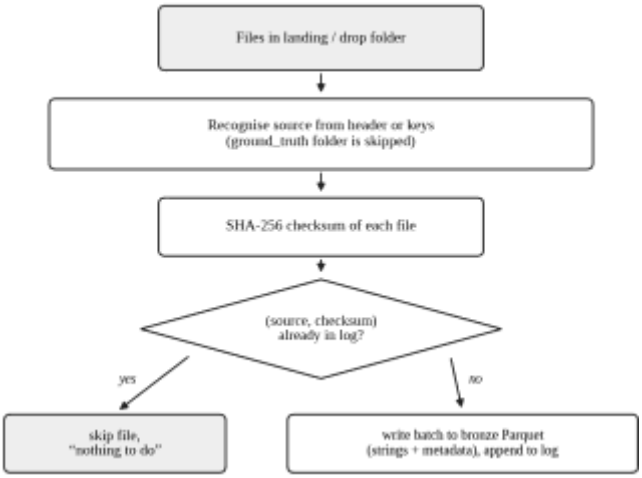


Fig. 5. Idempotent bronze ingestion: a file whose source and checksum are already logged is skipped.

D. Data quality

Quality rules are evaluated in Spark at load (Fig. 6). A row that fails a rule is moved to quarantine with its reason codes, and a per-load report is shown on the dashboard (DQ-2). The report of the SF = 0.10 load lists 16 rules (Table V): required columns and values, age range, sex, onset and report dates, date order, report delay, ICD-10 code, lab and admission flags, outcome, unique encounter id, registered facility, valid geography and reasonable values. Of 2,699,025 input rows, 2,553,555 are clean and 145,470 (5.4%) are quarantined.

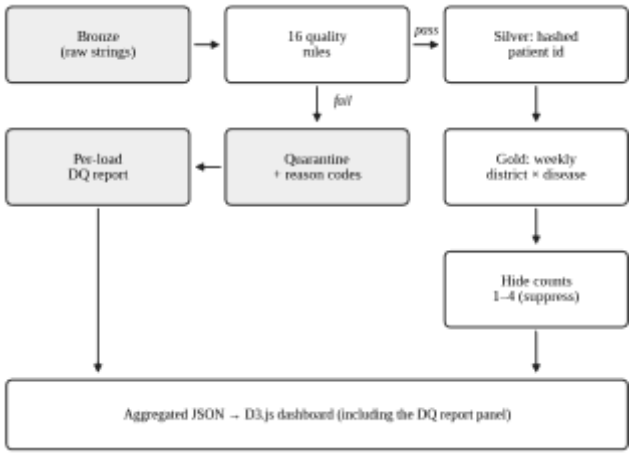


Fig. 6. Quality and privacy flow from bronze to the dashboard.

TABLE V. DATA-QUALITY RULES, SF = 0.10 LOAD

Rule	Passed	Failed
DQ_SCHEMA_REQUIRED_COLUMNS	2,699,025	0
DQ_REQUIRED_VALUES_NOT_NULL	2,699,025	0
DQ_AGE_IN_RANGE	2,680,063	18,962
DQ_SEX_ALLOWED	2,690,973	8,052
DQ_ONSET_DATE_VALID	2,699,025	0
DQ_REPORT_DATE_VALID	2,699,025	0
DQ_DATE_ORDER	2,699,025	0
DQ_REPORT_DELAY_0_21_DAYS	2,698,915	110
DQ_ICD10_ALLOWED	2,699,025	0
DQ_LAB_CONFIRMED_BINARY	2,699,025	0
DQ_ADMITTED_BINARY	2,699,025	0
DQ_OUTCOME_ALLOWED	2,699,025	0
DQ_ENCOUNTER_ID_UNIQUE	2,592,257	106,768
DQ_FACILITY_REGISTERED	2,685,648	13,377
DQ_GEOGRAPHY_VALID	2,685,648	13,377
DQ_REASONABLE_VALUES	2,680,063	18,962

The failures match what the generator injected (Fig. 7). Invalid ages and sexes agree with their injection rates (0.7% and 0.3% of 2,699,025 rows give about 18,900 and 8,100, against 18,962 and 8,052 found); the 13,377 unregistered facilities are the 13,107 orphan keys plus their copies in resubmitted rows; and the 106,768 repeated ids are twice the 53,384 resubmitted records, because both copies share an id. The 110 delays above 21 days are consistent with the back-filled reports of the silent facility.

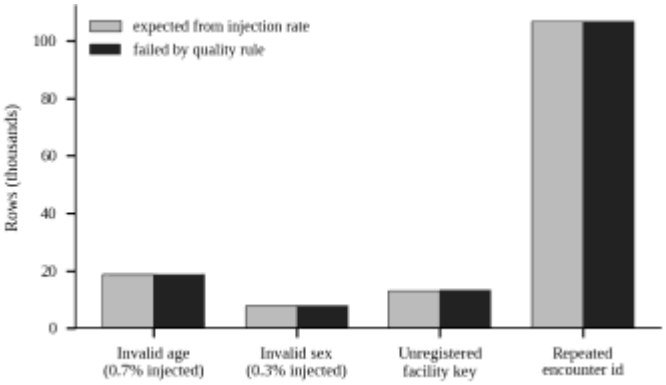


Fig. 7. Rows failing quality rules compared with the defects injected by the generator.

Known defect to fix. The clean count equals the input minus the quarantined rows, and the uniqueness rule fails both copies of a repeated id. Both copies are therefore quarantined, so about 53,000 distinct, valid encounters (about 2% of the data) are missing from silver and from the weekly counts. The rule should keep the first copy (earliest report date) and quarantine only the later copies. Until this is fixed, Table V and Fig. 7 describe the current behaviour.

E. Lineage

Every bronze row carries its file name, checksum and batch id, and the ingest log maps each batch to its files and row counts. Gold-level lineage to bronze batches has not yet been validated, so lineage is demonstrated down to silver and bronze files only.

F. Privacy

Patient identifiers are replaced by a salted hash before the cleaned silver output; the salt is kept in a local, git-ignored file and never committed. The browser receives only aggregated JSON, with counts of 1–4 hidden (Section II-A). For releases outside the Ministry we recommend adding Laplace noise to weekly district counts instead of relying on suppression alone, because suppression does not defend against linkage with outside data.

V. Analytics Methods

This section describes what the gold outputs and the dashboard show; the implementation is in the analytics folder of the repository.

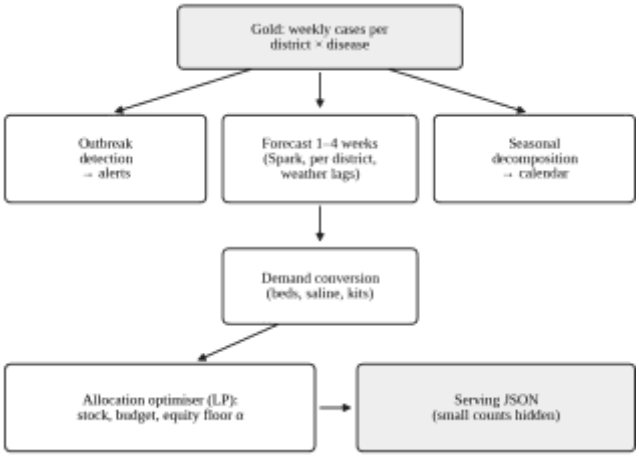


Fig. 8. Analytics flow: detection, forecasting, demand conversion, allocation and calendar, all reading the gold layer.

A. Outbreak detection (AN-1). A district–disease signal is raised when the recent weekly count is at least 2.0 standard deviations (adjustable) above its expected level and has grown by at least 50%. Flagged districts appear as alert markers on the map and in the ranked early-warning table, and the threshold can be changed with the what-if slider.

B. Forecasting (AN-4). Forecasts cover one to four weeks per district and disease and are trained in parallel with Spark; the dashboard describes them as rolling-origin forecasts with prediction intervals. Forecast quality is checked in a rolling-origin backtest (Section VI-B), and the dashboard shows the forecast median with an approximate 80% interval and the baseline alert threshold.

C. Demand conversion (AN-6). Case forecasts are converted into demand for resources such as IV saline units, NS1 test kits and ORS units; unit costs and stock levels are synthetic planning assumptions, as the dashboard states.

D. Allocation optimiser (AN-7). The optimiser minimises severity-weighted unmet demand across resource pools subject to national stock, a budget and an equity floor α times each district's population share, solved with PuLP/CBC. The budget is a share of the estimated replacement cost of the four-week forecast demand, α is the policy lever for equity, and the baseline plan shares the same budget by population.

E. Preparedness calendar (AN-9). Weekly counts per division and disease are decomposed into seasonal components to give, for the next year, when to begin campaigns and pre-position supplies. The calendar is shown by division, disease and month.

VI. Evaluation

A. Detection (AN-2, AN-3)

The injected outbreak list (33 outbreaks, 96 district-level events) is the ground truth for evaluating detection by sensitivity, false alerts per district-year and days to detection. That evaluation is not included in this version of the report.

B. Forecasting backtest (AN-5)

The dashboard reports the coverage of the 80% prediction intervals in a rolling-origin backtest (Table VI). Coverage is close to the nominal 80% at one week and rises above it at longer horizons, which means the intervals are wider than needed there. Point-error scores against the seasonal-naive baseline are not included in this version.

TABLE VI. 80% INTERVAL COVERAGE BY HORIZON

Horizon (weeks)	1	2	3	4
Coverage	81.3%	88.1%	90.5%	92.1%

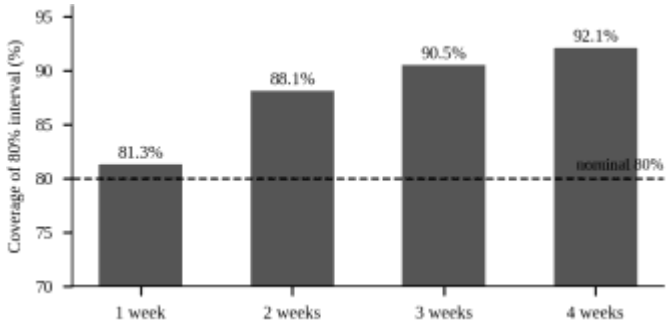


Fig. 9. Coverage of the 80% prediction intervals by forecast horizon (rolling-origin backtest).

C. Allocation versus baseline (AN-7, AN-8)

The dashboard compares the optimised plan with a population-proportional plan across budget levels from 20% to 100% of the replacement cost (Fig. 13). Numerical results are not reproduced in this version of the report.

D. Scalability and Spark optimisations (ENG-2, ENG-3)

Our laptop has 7.8 GiB of RAM and the assignment's scale test assumes 16 GB, so the sweep at SF = 0.25, 0.5 and 1.0 has not been run, and no result from this machine may be presented as meeting that target. The benchmark runner (pipeline.experiments.scale_benchmarks) checks memory and free disk, uses a temporary lake for each run and records per-stage timings and output sizes in reports/scale_benchmarks.json and .csv. An earlier SF = 1 sizing record covers synthetic encounter generation only. Table VII holds the numbers measured so far and the slots for the rest.

TABLE VII. SCALE EXPERIMENT

SF	Encounters	Generator (s)	Status
0.10	2,699,025	79–88	run on the 8 GB laptop
0.25	–	–	not run (needs 16 GB)
0.5	–	–	not run (needs 16 GB)
1.0	–	–	not run (needs 16 GB)

Hardware: laptop with 7.8 GiB of RAM and 8 threads, Spark 3.5.4 in local mode. SF = 1 must finish in 45 minutes or less on a 16 GB machine.

Two optimisations were measured with pipeline.experiments.spark_optimisations on 1,000,000 synthetic rows (three repeats): joining a small registry by broadcast was about 1.21 times faster than a sort-merge join, and partitioned Parquet reduced output size but gave no measurable scan-time gain in that run. These are smoke-test figures, not full-pipeline evidence. The same measurements on the full lake, with Spark UI plans, are not included in this version.

E. Incremental loads

Re-running ingestion on unchanged inputs added nothing (Section IV-C). Re-running a day in silver and gold, and the 14-day hidden-drop rehearsal, are not demonstrated in this version.

VII. DASHBOARD DESIGN AND USABILITY

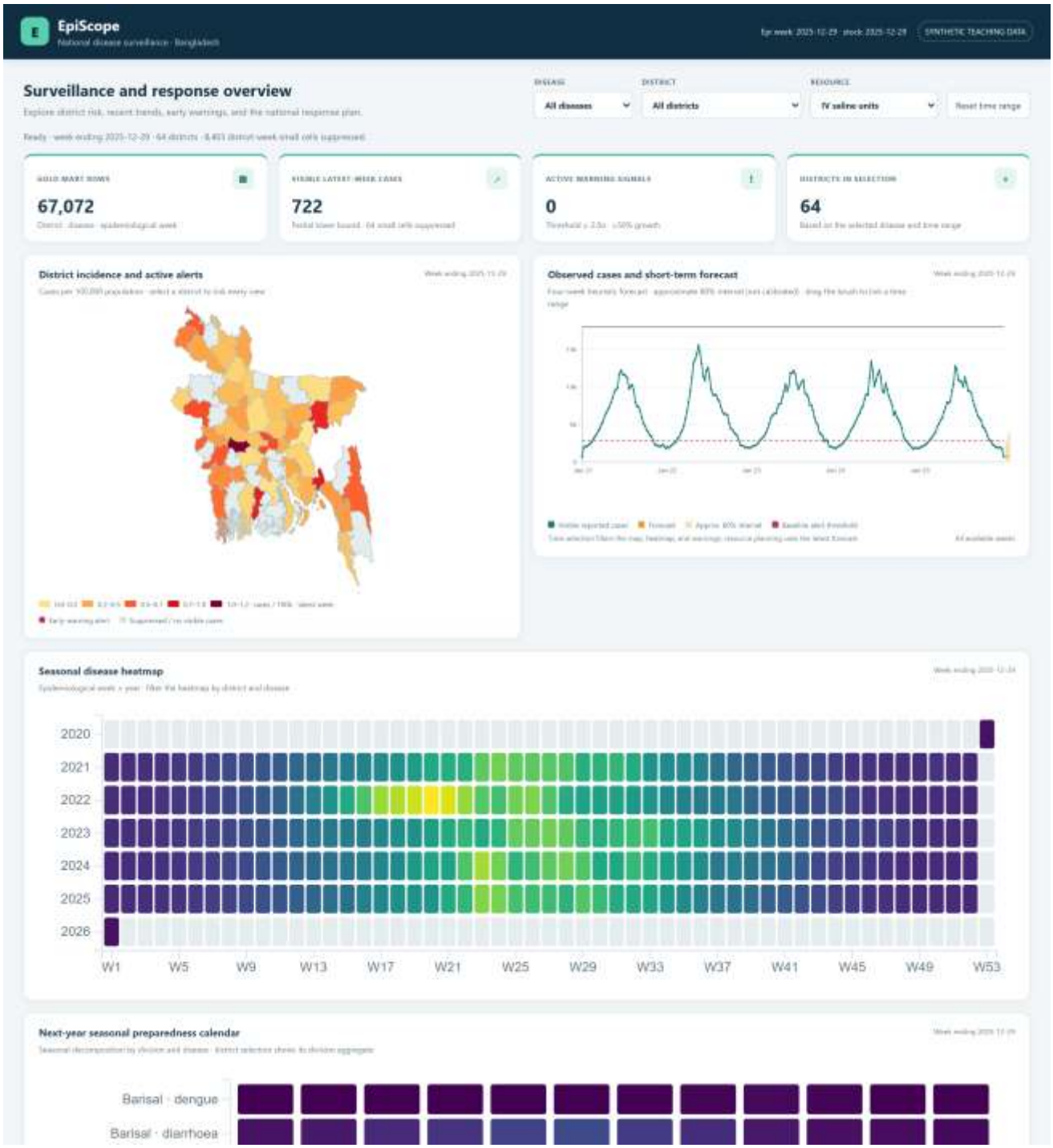
The dashboard is HTML with D3.js v7 and a simplified district boundary file, served by a local static server. It loads only aggregated, suppressed JSON, never line-level data. Figs. 10–13 show the national view, the preparedness calendar, the data-quality panel and the what-if controls.

Views and linking (VIS-2, VIS-3). The map shows incidence per 100,000 with alert markers and grey areas for suppressed cells; the time series shows reported cases, a four-week forecast with an approximate 80% interval, the alert threshold and a brush; further views are the next-year preparedness calendar (division × disease × month, from seasonal decomposition), the ranked early-warning table, and the allocation view that compares forecast demand, stock, recommended dispatch and the population baseline. Disease, district and resource selectors, the map and the brush update the other views and the summary cards. The seasonal heatmap view is the preparedness calendar (division × disease × month).

What-if panel (VIS-5). Sliders for national budget, equity floor α and alert threshold load precomputed optimiser scenarios and recompute alerts, with an efficiency–equity chart for the optimised and baseline plans.

Honesty and accessibility (VIS-6). Every panel shows its data date, a banner states that the data are synthetic, forecasts carry intervals, suppressed cells are marked, a note says forecasts are not operational predictions, tooltips explain hidden values (for example “Feni: one or more small cells suppressed”), and a data-quality panel lists each rule with its pass and fail counts.

Usability check. A usability test with at least two non-CS users has not been run yet. Rough edges seen in a recording of the dashboard: weeks with suppressed counts are drawn as zero in the time series (Feni), which can read as “no cases”; forecast demand and dispatch values below 5 are shown as “Suppressed (1–4)”, hiding planning information although they are estimates; a resource that does not apply to the chosen disease (for example NS1 kits for diarrhoea) leaves the allocation panel empty; and when the latest, still incomplete week has no alerts the ranking is empty, so a way to look at an earlier week would help.


EpiScope
Surveillance and response overview
67,072
722
0
64
District incidence and active alerts
Observed cases and short-term forecast
Seasonal disease heatmap
2020
2021
2022
2023
2024
2025
2026
W1
W5
W9
W13
W17
W21
W25
W29
W33
W37
W41
W45
W49
W53
Next-year seasonal preparedness calendar
Barisal · dengue
Barisal · diarrhoea



Silver data-quality report

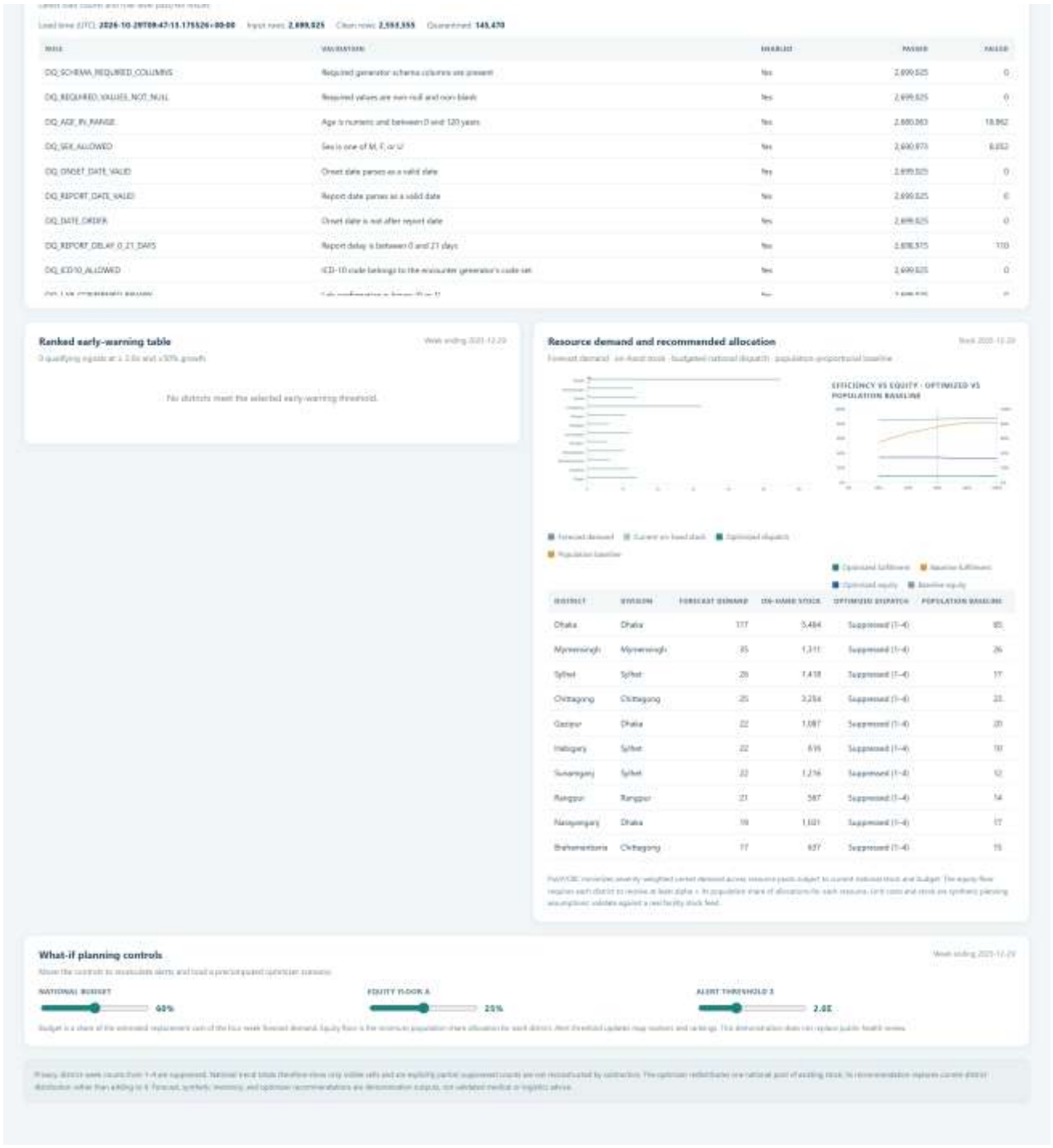


Fig. 4. EpiScope dashboard, national role (SF = 0.10 build). Views are linked: selecting a district on the map updates the time series, heatmap, table and allocation view

VIII. Ethical, Societal and Sustainability Considerations

Consequences of errors. A missed outbreak costs lives and a false alarm costs money and, repeated, the trust that makes officials act on the next alert, so both error types are reported and every alert should be confirmed with the district before a team moves. *Privacy and stigma.* Identifiers stop at bronze as salted hashes, the browser sees only aggregates, and small counts are hidden even for districts, because small communities (for example malaria in the hills) can be stigmatised.

Equity and misuse. Allocating only by expected unmet need concentrates supplies in the biggest cities; the equity floor makes the trade-off explicit and gives policymakers a number to defend. Alerts could be used to blame districts, so the interface should show reporting context (such as back-filled reports) and access should be role-based. Forecasts and plans are advisory, with intervals and a baseline plan to invite human judgement.

Honesty about data and AI use. Synthetic data can make methods look better than they will on real reports [4], so every result must be re-validated on Ministry data. We used AI coding assistants (GitHub Copilot and Claude, as listed in AI_USAGE.md) for first drafts of code and text; AI_USAGE.md lists tools, purposes and prompts, and each member is responsible for explaining the code in their area. Sustainability: the platform is open source, runs on a laptop and recomputes incrementally.

IX. Limitations and Future Work

(1) All encounter, facility and stock data are synthetic, so detection and forecast accuracy are optimistic; validation on real surveillance data is the first next step. (2) The scale sweep has not been run on a 16 GB machine, and the Spark optimisation evidence is from a small benchmark. (3) Storage is plain Parquet without ACID merge or time travel. (4) Some silver and gold steps still use pandas, tables are not registered in a documented Spark SQL catalog, stock snapshots (S6) were not ingested, late-record handling and gold-level lineage are not demonstrated, there is no streaming path or HDFS/MinIO storage, and the quantitative detection evaluation, the point-error scores of the forecasts and the numerical allocation results are not included in this version. (5) Population is a single 2023 estimate used for all five years. (6) The optimiser is single-period with linear costs; stock dynamics, lead times and uncertainty are future work. (7) Suppression protects the published hierarchy but not against outside data; formal differential privacy is recommended for public releases.

References

[1] L. Hutwagner, W. Thompson, G. M. Seeman, T. Treadwell, “The bioterrorism preparedness and response Early Aberration Reporting System (EARS),” J. Urban Health, 80(1), 2003.

[2] C. P. Farrington et al., “A statistical algorithm for the early detection of outbreaks of infectious disease,” J. R. Stat. Soc. A, 159(3), 1996.

[3] J. Bracher, E. L. Ray, T. Gneiting, N. G. Reich, “Evaluating epidemic forecasts in an interval format,” PLoS Comput. Biol., 17(2), 2021.

[4] D. Lazer, R. Kennedy, G. King, A. Vespignani, “The parable of Google Flu: traps in big data analysis,” Science, 343(6176), 2014.

[5] M. Zaharia et al., “Apache Spark: A unified engine for big data processing,” Commun. ACM, 59(11), 2016.

[6] M. Bostock, V. Ogievetsky, J. Heer, “D3: Data-Driven Documents,” IEEE TVCG, 17(12), 2011.

[7] L. Sweeney, “k-anonymity: A model for protecting privacy,” Int. J. Uncertainty, Fuzziness Knowl.-Based Syst., 10(5), 2002.

[8] World Health Organization, Technical Guidelines for Integrated Disease Surveillance and Response (IDSR), 3rd ed.

[9] Open-Meteo.com, “Historical Weather API,” https://open-meteo.com (accessed 2026).

[10] D. Runfola et al., “geoBoundaries: A global database of political administrative boundaries,” PLOS ONE, 15(4), 2020.

Appendix A. Policy Brief for the Director, Disease Control & Preparedness Unit

Seeing outbreaks sooner and sending supplies where they save most lives

Important: this prototype was tested on realistic simulated records for all 64 districts (2021–2025), because real patient data cannot be used for a student project. Numbers show what the system can do and must be re-checked on the Ministry's own data before any operational decision.

1. The situation. Information on dengue, diarrhoea and cholera, influenza-like illness and malaria reaches the Ministry late, in many formats, and is mostly studied after the fact. As a result, saline, test kits and teams can reach a district after its hospitals are already full. EpiScope brings hospital reports, weather, population and stock levels together and turns them into early warnings, forecasts for the next four weeks, a recommended distribution of supplies, and a calendar for the year ahead.

2. Three recommendations. (1) Hold a weekly warning review using the ranked list. A district is flagged when its weekly count is well above what is expected (at least 2.0 standard deviations) and has grown by at least half; each warning should be confirmed by a phone call to the district before a team is sent. (2) Distribute supplies with the recommended plan, which guarantees every district a minimum share instead of sharing only by population; the sliders show what a smaller budget or a larger guarantee would change. (3) Follow the preparedness calendar. In the simulated data for Dhaka division, dengue rises in the middle of the year and peaks around August and September, diarrhoea peaks around May and June, and influenza-like illness around June to August, so prevention and supplies should be in place some weeks earlier. Numerical estimates of the impact are not included in this version.

3. How to read the dashboard. On the map, darker means more cases per 100,000 people in the chosen weeks; a red marker shows a district with an active warning; grey areas hide small numbers for privacy. In the weekly chart the solid line is what has been reported, the shaded fan is the forecast (darker is more likely) and the dotted line is the warning level. In the warning list, districts at the top need attention first. In the supplies view compare expected need, current stock and the recommended shipment; the sliders let you ask what happens if the budget is smaller or if every district is guaranteed a larger minimum share. Every panel shows the date of its data.

4. Confidence and limitations. Forecasts one week ahead stayed inside their stated range about as often as promised (81 times in 100 for an 80% range), but ranges for three and four weeks ahead were wider than needed (about 90 and 92 in 100), so longer forecasts should guide preparation, not exact orders. Forecast error compared with a simple “same as last year” method is not included in this version. Slow-building outbreaks are harder to spot than sudden ones, unusual weather causes real rises that officials must interpret, results depend on hospitals reporting regularly, and supply needs rest on standard assumptions that should be confirmed with clinicians and store staff.

5. Data protection. Names and ID numbers never leave the secure first storage area: they are replaced by an irreversible code made with a secret key kept outside the software. The dashboard shows only totals per district and week, and any count from 1 to 4 is hidden.

Appendix B. Contribution Matrix

L = lead, S = support, – = none. The cells that are empty must be completed by the members themselves, with the percentage shares (adding to 100) agreed and signed by all three, and must reflect who actually did each piece of work. The commit counts below come from *git shortlog -sn origin/main* on 25 Oct 2026 and must be refreshed before submission. The matrix must agree with the Git history and AI_USAGE.md.

Work area	Eva	Naima	Shreoshee
Problem analysis, proposal, conflicts	L		
Generator and public data	L		
Ingestion, lake layers, incremental loads	L		
Quality, lineage, privacy controls	L		Serving privacy
Outbreak detection and evaluation		L	
Forecasting, backtest, demand		L	
Optimiser and calendar		L	
Serving layer and D3.js dashboard			L
Scalability and Spark optimisations	L		
Report and policy brief	Integration, architecture	Analytics sections	Dashboard sections
Video production	Edit: Chapter 1-3	Chapter 4	Chapter 5-6
Agreed share σ (%)			
Commits on main	13	2	3
Signature			

Appendix C. Individual Reflections

Eva Kalam, Data Engineer.

I led the generator and the first layers of the pipeline: the facility registry, the weather download, the daily and encounter-level generators, and the bronze ingestion with checksums. The hardest problem was making loads repeatable. A file can arrive twice, a record can be resubmitted, and a late report can change an old week; the checksum log makes a second ingest do nothing, and the data-quality report showed that the injected duplicates, orphan keys and invalid values were caught, and it also exposed a defect in my duplicate rule, which quarantined both copies of a repeated id. Setting up Spark on Windows with an 8 GB laptop and slow internet cost time, and it taught me to check Java, memory and disk before running anything. One decision I would revisit is that the full scale sweep still needs a 16 GB machine, which I should have arranged earlier. I used AI assistants for first drafts and for explanations, as AI_USAGE.md records, and I am responsible for explaining every part of my area.

Aklima Akter Naima, Analytics Engineer.

I focused on analytics: outbreak detection, weekly forecasting, and resource-planning scenarios. The detector results taught me that sensitivity alone is not enough; detecting almost all labelled synthetic outbreaks is not useful if the alert volume is too high for district teams. Comparing ridge forecasts with a seasonal-naive baseline gave a more meaningful picture than reporting model error alone, while horizon-specific interval coverage showed that uncertainty still needs attention. The allocation results were a reminder to treat unexpected output as a diagnostic finding, not to present a model as successful because it uses an optimizer. I would reconcile detector summaries, confirm which forecast appears in the dashboard, audit allocation equations and units, and validate methods on data beyond the synthetic generator.

Shreoshee Chowdhury Oma, Visualisation Engineer.

I focused on presenting EpiScope through the dashboard: the district map, case trends, seasonal views, data-quality information, warning candidates, and resource-planning panel. The work highlighted how interface choices affect interpretation. A suppressed cell is not a zero, an empty warning panel does not mean no candidates exist in the export, and a forecast needs its method and uncertainty explained beside the graph. The dashboard snapshot made the data date and synthetic-data notice important parts of the experience, not decorative labels. If I continued, I would test the dashboard with intended users, check accessibility and small-screen behavior, and ensure charts use the same validated model and metric definitions as evaluation artifacts. I would keep limitations visible wherever a user might interpret a scenario as an operational recommendation.

APPENDIX D. REQUIREMENT STATUS

Status is taken from the repository README, the dashboard recording and our own runs on 25 Oct 2026. Update it before submission.

Requirement	Status
ING-1 Bronze + metadata	Done: 65 files, SHA-256 checksums, batch ids
ING-2 Spark for heavy work	Partial: bronze in Spark; some silver/gold use pandas
ING-3 Partitioned, catalog	Partial: silver Parquet by epi-year × division; catalog registration not documented
ING-4 Incremental, idempotent	Partial: bronze done; silver and gold late records not demonstrated
ING-5 HDFS / MinIO	Not done
ING-6 Streaming	Not done
ING-7 Delta time travel	Not done (optional)
DQ-1 ≥12 rules, quarantine	Done: 16 rules, quarantine with reason codes; duplicate rule to be fixed (Section IV-D)
DQ-2 Per-load DQ report	Done: dashboard panel
DQ-3 Lineage to bronze	Partial: file level; gold level not validated
DQ-4 Cross-facility dedup	Not evidenced
PRV-1 Salted hash	Done: salted hash, salt kept outside the repository
PRV-2 Small-cell suppression	Partial: counts 1–4 hidden; subtraction audit not done
PRV-3 Role views	Not evidenced
PRV-4 k-anonymity, DP note	Not done (optional)
AN-1/2/3 Detection, evaluation	Alerts shown on the dashboard; evaluation against ground truth not included
AN-4 Forecast, Spark parallel	Implemented; interval coverage 81.3–92.1%
AN-5 Backtest vs naive	Partial: interval coverage; MAE/WIS not included
AN-6 Demand conversion	Shown on dashboard; assumptions are synthetic
AN-7/8 Optimiser, equity curve	Implemented (PuLP/CBC); numerical results not included
AN-9 Preparedness calendar	Implemented (division × disease × month)
AN-10 SEIR	Not evidenced (optional)
VIS-1 D3.js only	D3.js dashboard; folder not yet in the repository
VIS-2 Five linked views	Map, time series, allocation, ranking and calendar heatmap seen; not yet pushed
VIS-3 Linked interaction	Seen in recording
VIS-4 ≤10 MB, aggregates	Aggregates only; size not measured
VIS-5 What-if panel	Seen in recording
VIS-6 Accessibility, honesty	Partial: banner, dates, bands, suppression
VIS-7 Mobile / playback	Not evidenced (optional)
ENG-1 One command	Partial: run_pipeline.bat sets up the environment and runs the stages; analytics and dashboard still to merge
ENG-2 Scale 0.25/0.5/1	Not done: needs a 16 GB machine
ENG-3 Two optimisations	Partial: microbenchmark; Spark UI evidence missing
ENG-4 Tests, DAG + retries	Partial: 49 tests; DAG with retries not evidenced
H1 Code, tag v1.0, private	Open: tag pending; repository must be private
H2 Report PDF	This document
H3 Video 15–20 min	Not recorded